

Planax: A JAX-Accelerated High-Fidelity Reinforcement Learning Benchmark for Large-Scale Fixed-Wing Aerial Swarms

Chuangyi Xue, Qinyu Dong, Qihan Liu, Jun Yang, Bin Liang, Yang Qi, Xinyao Qin, Jinsheng Ren, and Zhiyang Zhou

Abstract—Multi-agent reinforcement learning (MARL) for fixed-wing aerial robots currently faces a critical bottleneck: the severe trade-off between physical realism and training throughput. Existing open-source platforms often rely on simplified point-mass kinematics, failing to capture the non-holonomic constraints and energy maneuverability of highly dynamic flights. Conversely, traditional high-fidelity aviation simulators suffer from severe computational bottlenecks, rendering large-scale parallel MARL infeasible. To address this, we propose Planax, a JAX-accelerated, fully GPU-resident high-fidelity simulation framework. By leveraging XLA compilation and operator fusion, Planax integrates a nonlinear six-degree-of-freedom (6-DOF) flight dynamics model—driven by authentic aerodynamic wind-tunnel data via branch-free tensor operations—directly into optimized GPU kernels. Comprehensive experiments demonstrate that Planax achieves an unprecedented simulation throughput exceeding 20 million steps-per-second on a single NVIDIA A100 GPU and supports up to 10^7 concurrent 6-DOF agents. The framework successfully stabilizes massive decentralized formation keeping and hierarchical pursuit-evasion benchmarks. Crucially, a rigorous sim-to-real proxy ablation reveals that policies trained without our high-fidelity constraints exploit physical loopholes, suffering a 100% crash rate due to structural overloads during agile maneuvers. In contrast, Planax ensures the generation of safe, physically grounded strategies. The Planax codebase and video demonstrations are open-source and available at: <https://anonymous.4open.science/r/Aeroplanax-2087>.

Index Terms—Aerial Systems: Mechanics and Control, Multi-Agent Reinforcement Learning, Swarm Robotics, GPU-Accelerated Simulation, High-Fidelity Benchmarking.

I. INTRODUCTION

THE deployment of large-scale multi-agent reinforcement learning (MARL) for autonomous aerial systems is rapidly transitioning from conceptual exploration to physical implementation. While MARL has demonstrated remarkable success in coordinating complex multi-robot behaviors, its application to high-dynamic fixed-wing aerial robots remains severely constrained by a fundamental “sim-to-real” gap. Unlike hovering quadrotors, fixed-wing flight is dominated by strict non-holonomic constraints, complex rigid-body coupling, and severe energy maneuverability limitations. Existing MARL abstractions often simplify these vehicles into

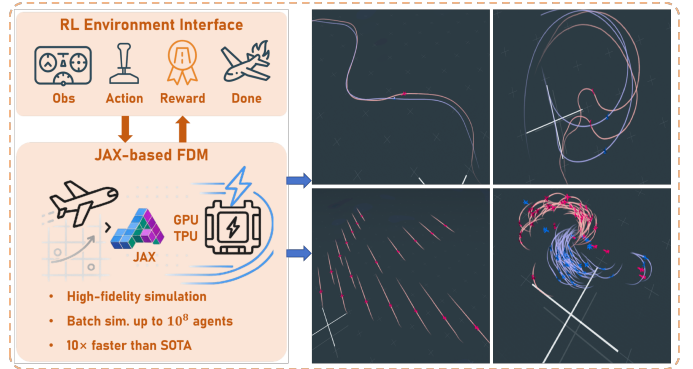


Fig. 1. **Overview of the Planax framework.** (Left) The JAX-accelerated simulation architecture and reinforcement learning (RL) environment interface. (Right) Tacview visualization of the training results, demonstrating single-agent high-dynamic S-maneuvers, 2v2 adversarial combat, large-scale (50v50) swarm dogfights, and multi-agent cooperative formation flight.

kinematic point-masses. Such simplifications allow policies to exploit non-physical loopholes during training, rendering the learned cooperative or adversarial strategies undeployable in the real world. Overcoming this gap necessitates training environments that enforce rigorous physical realism.

However, achieving such high-fidelity realism introduces a critical computational bottleneck for MARL, which typically requires millions to billions of environmental interactions. Currently, the simulation ecosystem presents a severe dichotomy. On one hand, widely adopted robotics RL frameworks (e.g., Gym-PyBullet-Drones, Isaac Gym) excel at massively parallel, GPU-accelerated execution but are predominantly designed for ground robots or quadrotors, lacking the infrastructure for sophisticated aerodynamic modeling. On the other hand, traditional high-fidelity aviation simulators (e.g., JSBSim) provide validated six-degree-of-freedom (6-DOF) flight dynamics driven by authentic wind-tunnel data. Yet, these engines are strictly CPU-centric. The resultant host-device memory transfer overheads and fragmented sequential execution make them computationally prohibitive for large-scale parallel policy optimization.

To bridge the gap between simulation throughput and physical fidelity, we propose Planax, a massively parallel, JAX-accelerated simulation framework tailored for large-scale fixed-wing aerial swarms. As illustrated in Fig. 1, Planax eschews the conventional “CPU-simulator paired with GPU-policy” paradigm. Instead, it leverages XLA (Accelerated

Chuangyi Xue, Qinyu Dong, and Qihan Liu contributed equally to this work.

The authors are with the Institute of Navigation and Control, Department of Automation, Tsinghua University, Beijing 100084, China (e-mail: xcy22@mails.tsinghua.edu.cn; dqy24@mails.tsinghua.edu.cn; liuqh.thu@gmail.com; yangjun603@tsinghua.edu.cn; bliang@tsinghua.edu.cn).

Jun Yang and Bin Liang are the corresponding authors.

Manuscript received XX XX, 202X; revised XX XX, 202X.

Linear Algebra) compilation and operator fusion to integrate a rigorous 6-DOF nonlinear flight dynamics model directly into GPU-resident SIMD kernels. This end-to-end device residence completely eliminates PCIe memory transfer latencies. Most distinctively, Planax processes multi-dimensional authentic aerodynamic data through highly optimized, branch-free tensor operations at the XLA operator level, avoiding the physical distortion of surrogate approximations while maintaining maximum parallel efficiency.

The main contributions of this letter are threefold:

- We introduce the first JAX-based, fully GPU-resident MARL framework specifically designed for high-dynamic fixed-wing aerial robots, achieving an unprecedented simulation throughput exceeding 2.3×10^7 steps-per-second (SPS) on a single NVIDIA A100 GPU and supporting up to 10^7 concurrent 6-DOF agents.
- We pioneer a native tensorized implementation of complex aerodynamic lookup tables (LUT) directly within XLA-fused kernels. This architecture preserves bit-for-bit physical consistency with real-world aerodynamic data while avoiding the SIMD branch divergence typical of traditional lookup operations.
- We provide an open-source benchmark suite encompassing decentralized formation keeping and adversarial maneuvering. Through rigorous sim-to-real proxy ablations, we demonstrate that incorporating strict high-fidelity aerodynamic constraints is essential for robust policy transfer, and we validate the framework's compatibility with mainstream MARL algorithms.

II. RELATED WORK

A. Aerial Robot Simulators

The advancement of reinforcement learning (RL) for aerial robots is intrinsically linked to the evolution of simulation fidelity and throughput. Traditional high-fidelity flight simulators, most notably JSBSim [1] and X-Plane [2], provide rigorous nonlinear six-degree-of-freedom (6-DOF) dynamics driven by authentic wind-tunnel data. While these engines ensure physical credibility, their CPU-centric serial execution models create a severe computational bottleneck, making it impractical to generate the billions of interaction steps required by modern multi-agent reinforcement learning (MARL) algorithms at scale.

To overcome these limitations, the robotics community has pivoted toward GPU-accelerated platforms. Frameworks such as *Gym-PyBullet-Drones* [3] and *Isaac Gym* [4] leverage hardware acceleration to achieve massive simulation throughput. However, these environments are predominantly optimized for hovering quadrotors or simplified point-mass kinematics. They frequently lack the specialized aerodynamic primitives—such as multi-dimensional lift/drag look-up tables (LUTs) and complex inertial coupling—necessary to capture the non-holonomic constraints and energy maneuverability characteristic of high-dynamic fixed-wing flight.

Planax occupies a unique ecological niche by unifying these paradigms. As summarized in Table I, while existing platforms compromise on either multi-agent scalability, physical

fidelity, or execution efficiency, Planax satisfies all constraints. Building upon the foundational concept of parallelized flight dynamics introduced by *NeuralPlane* [5], Planax extends the capability into the multi-agent regime. By integrating native tensorized aerodynamic operations within JAX-accelerated ecosystems [6], [7], [8], Planax eliminates the sim-to-real gap without sacrificing training efficiency.

B. MARL Benchmarks for High-Dynamic Systems

Standard MARL benchmarks, such as the Multi-Agent Particle Environment (MPE) [9] and the StarCraft Multi-Agent Challenge (SMAC) [10], have been instrumental in the development of cooperative and competitive algorithms. Nevertheless, these testbeds primarily focus on low-dimensional grid-worlds or kinematic navigation, lacking continuous-time, physics-consistent aerodynamic constraints that dictate the feasibility of aerial maneuvers.

Recent aeronautics-oriented studies have begun emphasizing cooperative decision-making in more realistic settings, proposing platforms like QPlane [11] and various hierarchical combat environments [12], [13]. However, these tools are often limited by scalability or lack end-to-end device residence. In contrast, Planax provides a native, large-scale MARL infrastructure that supports high-dynamic pursuit-evasion and dense formation tasks under strict flight-envelope constraints, establishing a new standard for evaluating decentralized coordination in realistic aerial environments.

III. SYSTEM ARCHITECTURE

This section describes the architectural design of Planax (depicted in Fig. 2), focusing on how it reconciles high-fidelity aerodynamic modeling with the massive throughput requirements of multi-agent reinforcement learning (MARL).

A. GPU-Resident Flight Dynamics

To bridge the sim-to-real gap, Planax implements a nonlinear six-degree-of-freedom (6-DOF) flight dynamics model that strictly adheres to the physical laws of high-dynamic flight. By executing the entire dynamics chain on the GPU, we minimize the latency typically associated with heterogeneous simulation-learning loops.

1) *Singularity-Free Rigid-Body Kinematics*: High-dynamic maneuvers, such as the S-maneuvers used in our agile control benchmarks, frequently involve extreme attitude changes where traditional Euler angles encounter gimbal lock singularities. To ensure numerical stability and robustness,

TABLE I
COMPARISON OF AERIAL SIMULATION PLATFORMS FOR MARL

Platform	Native MARL	GPU / JIT	Authentic Aero.	Device-Resident
X-Plane [2]	No	No	Yes (Standard LUT)	No (CPU)
JSBSim [1]	No	No	Yes (Standard LUT)	No (CPU)
Isaac Gym [4]	Yes	Yes	No (Point-mass)	Yes (Tensor API)
NeuralPlane [5]	Limited	Yes	Yes (MLP Surrogate)	Yes (Eager Tensors)
Planax (Ours)	Yes	Yes (JAX)	Yes (Tensor LUT)	Yes (XLA Fused)

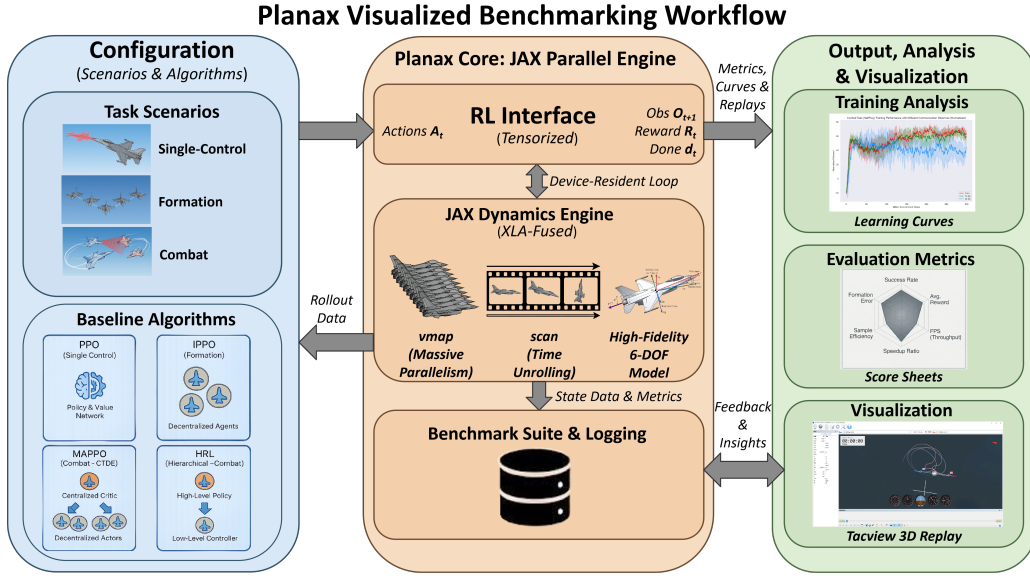


Fig. 2. **Overall benchmarking workflow in Planax.** The system architecture features a **fully device-resident** loop where both the 6-DOF flight dynamics and MARL policy optimization are compiled into a unified computational graph via XLA. This design ensures that all intermediate states remain in GPU VRAM, effectively eliminating the frequent host-device (CPU-GPU) memory transfer overheads common in traditional simulation-learning pipelines.

Planax adopts a unit quaternion representation [14] $\mathbf{q} = [q_0, q_1, q_2, q_3]^T$ for aircraft orientation. The kinematic rate of change is computed as (Eq. 1):

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}_b \end{bmatrix} \quad (1)$$

where $\boldsymbol{\omega}_b$ is the angular velocity in the body frame and $\otimes$ denotes quaternion multiplication. This formulation is implemented using JAX-native array operations, allowing the state of millions of agents to be updated concurrently via `vmap`-based vectorization.

2) *Branch-Free Tensorized Aerodynamic LUTs*: The fidelity of fixed-wing simulation is primarily dictated by the accuracy of the aerodynamic forces ($\mathbf{F}_{aero}$) and moments ($\mathbf{M}_{aero}$). Unlike simplified simulators that rely on linear approximations, Planax utilizes authentic aerodynamic coefficients derived from wind-tunnel data (e.g., NASA F-16 database).

A major technical challenge in porting such high-fidelity models to the GPU is the handling of multi-dimensional lookup tables (LUTs). Standard interpolation algorithms typically involve complex branching logic (`if-else` statements), which causes severe *branch divergence* and performance degradation on SIMD architectures. To maintain maximum throughput, we developed a `**native tensorized LUT engine**`. We re-sample the irregular wind-tunnel data into structured multi-dimensional grids across the flight envelope (e.g., angle of attack α , sideslip β , and control surface deflections δ). During runtime, Planax performs interpolation using pure tensor indexing and weighted summations:

$$C_x(\alpha, \beta, \delta) = \sum_i w_i \cdot \text{Grid}_x[\text{indices}_i] \quad (2)$$

This architecture ensures that the non-linear “hard” physics—including stall characteristics and cross-coupling

effects—are preserved without sacrificing computational efficiency. As demonstrated in Fig. 3, while neural approximations (e.g., NeuralPlane’s MLP surrogate) suffer from visible boundary degradation and systemic bias at high angles of attack, our Tensor LUT strictly maintains bit-for-bit mathematical equivalence with the validated wind-tunnel data. This zero-error physical grounding is the fundamental prerequisite for preventing policies from exploiting non-physical loopholes during aggressive envelope expansion.

B. Parallelization Architecture and JAX Acceleration

The fundamental bottleneck in conventional aerial MARL stems from the structural separation between the CPU-based simulator and the GPU-based policy optimizer. As formalized in our prior analysis, the wall-clock time for a single simulation-learning step can be decomposed as follows (Eq. 3):

$$T_{\text{step}} \approx T_{\text{H2D}} + T_{\text{D2H}} + N_{\text{launch}} \cdot T_{\text{launch}} + T_{\text{mem}} + T_{\text{compute}} \quad (3)$$

where T_{H2D} and T_{D2H} are host-to-device and device-to-host PCIe transfer latencies, N_{launch} is the number of GPU kernel launches, and T_{mem} is the global memory access time. Planax achieves massive acceleration by systematically driving $T_{\text{H2D}}, T_{\text{D2H}} \rightarrow 0$ and $N_{\text{launch}} \rightarrow 1$.

1) *Massive Concurrency via `vmap`*: Traditional simulators parallelize environments using multiprocessing, which introduces substantial Inter-Process Communication (IPC) overhead. Planax natively bypasses this by leveraging JAX’s `vmap` primitive (Algorithm 1, Line 7). By expressing the entire 6-DOF dynamics, aerodynamic evaluations, and reward logic in pure functional forms, `vmap` automatically transforms the single-agent transition function into a highly efficient, batched SIMD (Single Instruction, Multiple Data) operation.

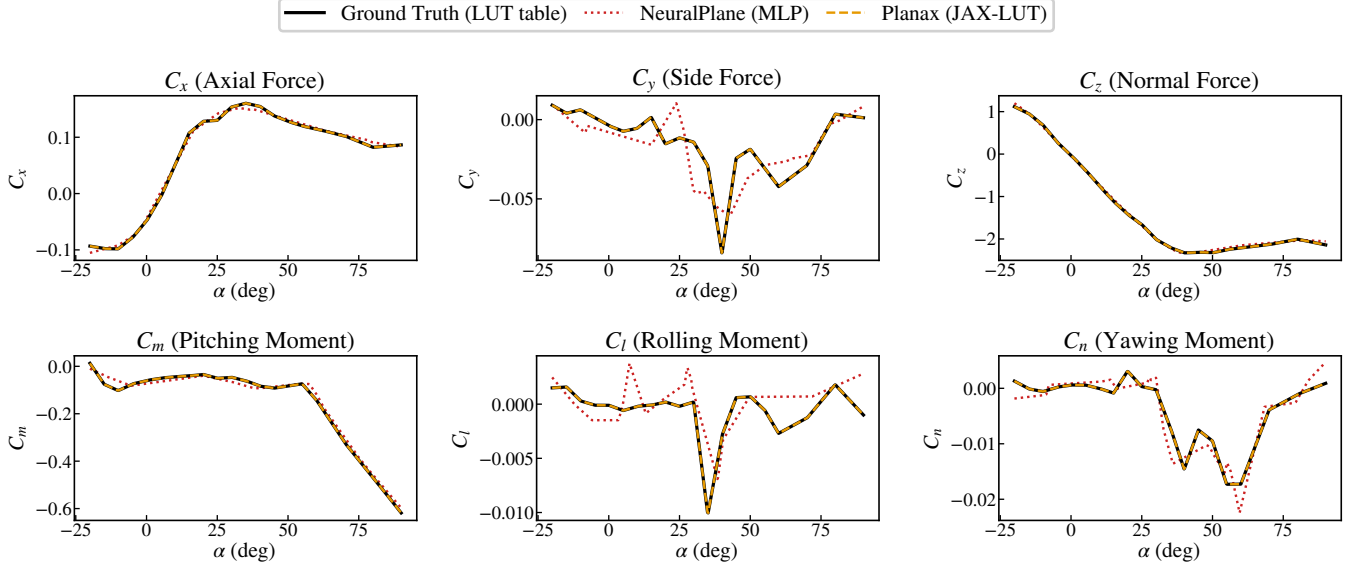


Fig. 3. **Aerodynamic Fidelity Evaluation.** Comparison of the six F-16 aerodynamic coefficients (C_x , C_y , C_z , C_l , C_m , C_n) computed by Planax’s branch-free JAX-LUT, NeuralPlane’s PyTorch MLP surrogate, and the Ground Truth (JSBSim/NASA tabular data) across varying angles of attack α (under fixed conditions: sideslip $\beta = 0^\circ$, elevator deflection $\delta_e = 0^\circ$). While the MLP surrogate exhibits noticeable systemic bias and boundary degradation at extreme maneuvers (e.g., high α stall regions in C_z and C_m), Planax maintains bit-for-bit mathematical equivalence with the ground truth, effectively eliminating aerodynamic approximation errors.

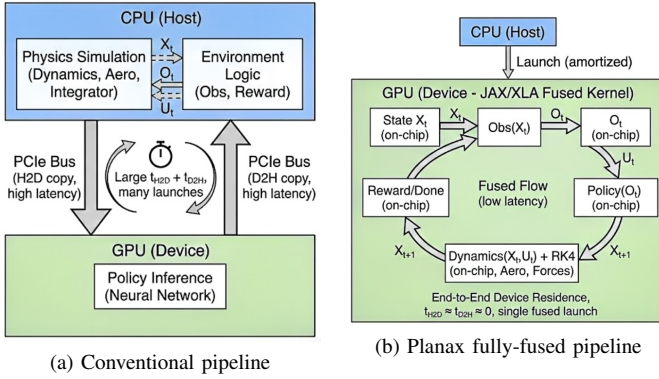


Fig. 4. **End-to-end throughput bottlenecks.** (a) The conventional MARL pipeline suffers from severe PCIe memory transfer latencies (T_{H2D} , T_{D2H}) and fragmented kernel launches. (b) Planax leverages XLA compilation to fuse the 6-DOF dynamics, **aerodynamic Tensor LUTs**, and reward logic into a single GPU-resident kernel, eliminating transfer overheads.

This allows the state updates for millions of agents to be executed concurrently across the GPU hardware grid without any Python-level loop overhead.

2) *XLA Operator Fusion and Loop Unrolling*: In complex physical simulations involving RK4 integration and multi-dimensional aerodynamics, sequentially executing fragmented operations leads to excessive global memory read/write latencies (T_{mem}). Planax utilizes `jax.lax.scan` to unroll the temporal simulation loop and applies Just-In-Time (`jit`) compilation powered by the XLA compiler. As illustrated in Fig. 4, XLA analyzes the entire computational graph and fuses the physics equations into a single, highly optimized GPU kernel (Algorithm 1, Line 11). Unlike PyTorch-based aerial simulators (e.g., NeuralPlane) that suffer from severe

Algorithm 1 End-to-End JAX Compilation Pipeline in Planax

Require: Base dynamics $f_{6DOF}(\mathbf{x}, \mathbf{u})$, Aero surrogate MLP_{aero}

Ensure: Fused rollout kernel $Rollout_{jit}$

1: **Define single-agent step (Pure Function):**

2: $\mathbf{C}_{aero} \leftarrow MLP_{aero}(\alpha, \beta, \delta)$

3: $\mathbf{x}_{t+1} \leftarrow RK4_Integrate(f_{6DOF}, \mathbf{x}_t, \mathbf{C}_{aero})$

4: $o_{t+1}, r_t, d_t \leftarrow TaskLogic(\mathbf{x}_{t+1})$

5: $step_1(\mathbf{x}_t, \mathbf{u}_t) \mapsto (\mathbf{x}_{t+1}, o_{t+1}, r_t, d_t)$

6: **Batch Vectorization (Agent Axis):**

7: $Step_{batch} \leftarrow jax.vmap(step_1, in_axes = (0, 0))$

8: **Time Unrolling (Horizon Axis):**

9: $Rollout \leftarrow jax.lax.scan(Step_{batch}, \mathbf{X}_0, \mathbf{U}_{1:T})$

10: **XLA Operator Fusion:**

11: $Rollout_{jit} \leftarrow jax.jit(Rollout)$

12: **return** $Rollout_{jit}$ executed entirely on GPU VRAM

Python-level dispatch overheads and fragmented global memory accesses during eager tensor execution, Planax’s compiled architecture executes the entire complex aerodynamic and kinematic chain as a monolithic device operation.

3) *Zero-Transfer Simulation-Learning Loop*: Because both the environment rollout and the MARL policy updates (e.g., PPO/MAPPO) are compiled into unified XLA computational graphs, experience replay buffers, trajectory sampling, and gradient descent all occur entirely within the GPU VRAM. This “zero-transfer” architecture eliminates the PCIe bottleneck, shifting the scalability limits directly to the raw compute and memory capacity of the GPU.

C. Task Environments and Benchmark Suite

Planax provides a standardized suite of tasks designed to evaluate varying levels of coordination complexity under strict flight-envelope constraints. By framing these environments around a hierarchical control scheme, the MARL policy network acts as a high-level planner, outputting reference commands (e.g., target heading, pitch, and velocity) which are subsequently tracked by low-level continuous flight controllers.

1) *Decentralized Formation Keeping*: In this cooperative task, a swarm of aerial robots must autonomously maintain a predefined geometric configuration (e.g., a V-shape or grid) while navigating toward a dynamic target point. The environment is formulated as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP).

- **Observation Space $\mathcal{O}$** : Includes the normalized ego-state (airspeed, attitude, and angular rates) alongside the relative displacement and velocity vectors to the K -nearest neighbors within a defined communication radius.
- **Action Space $\mathcal{A}$** : Continuous reference commands $[\psi_c, \theta_c, V_c]$ representing desired heading, pitch, and airspeed, scaled to $[-1, 1]$.
- **Objective & Termination $\mathcal{T}$** : Agents aim to minimize the Euclidean tracking error to their assigned geometric slots. The episode terminates early with a penalty if any agent violates safety constraints (e.g., aerodynamic stall, extreme overloads), collides, or deviates beyond a safe operational radius.

2) *Adversarial Pursuit-Evasion*: This benchmark abstracts multi-agent combat into a fundamental competitive interaction between pursuer and evader swarms. Operating under identical flight dynamics, the task focuses heavily on complex spatial coordination and relative positioning rather than extreme aerodynamic envelope expansion.

- **Observation Space $\mathcal{O}$** : A range-limited vector encompassing the ego-state, augmented with the relative states—specifically line-of-sight (LOS) angles and distances—of both allied and adversarial agents within the sensor’s field of view.
- **Action Space $\mathcal{A}$** : High-level trajectory commands identical to the formation task, processed by the underlying hierarchical tracking controller to execute aggressive bank-to-turn maneuvers.
- **Objective & Termination $\mathcal{T}$** : The primary objective is to reach and maintain an optimal trailing position behind the adversarial agents. The task error is defined geometrically by the distance from the pursuer’s forward axis (nose) to the evader’s rear axis (tail), minus a safety margin. Episodes terminate upon successful capture (proximity threshold met within the tail cone), an out-of-bounds violation, or an unintended departure from controlled flight due to aggressive command inputs.

Note: Detailed mathematical formulations of the dense reward shaping, exact state/action dimensions, and hyper-parameter configurations for all tasks are provided in the Supplementary Material.

IV. EXPERIMENTS AND BENCHMARKING

The empirical evaluation of Planax is designed to answer four critical questions regarding its efficacy as a MARL infrastructure: (1) Does the GPU-resident architecture yield massive throughput advantages over traditional simulators? (2) Does the tensorized aerodynamic engine preserve strict physical fidelity? (3) Is high-fidelity modeling strictly necessary for agile maneuvers (Sim-to-Real proxy)? (4) Can the framework successfully stabilize large-scale MARL training? All computational benchmarks and RL training are conducted on a single NVIDIA A100 (80GB VRAM) GPU and an AMD EPYC 64-core CPU.

A. Simulation Throughput and Scalability

To quantify the computational efficiency of Planax, we benchmark its simulation throughput—measured in Steps Per Second (SPS)—and memory footprint against JSBSim, the industry-standard high-fidelity aviation simulator. JSBSim is evaluated under both single-thread and heavily parallelized multi-process CPU configurations.

1) *Throughput Analysis (SPS)*: As illustrated in Fig. 5 (Left), traditional CPU-centric simulators exhibit severe scaling limitations. While multi-processing provides a linear speedup for small populations, the throughput of JSBSim rapidly saturates due to context-switching overhead and Inter-Process Communication (IPC) bottlenecks, peaking at approximately 2×10^3 SPS even on a 64-core CPU.

Conversely, Planax leverages `vmap` vectorization to achieve massive SIMD parallelism. For small agent populations ($N < 10^3$), the throughput is primarily bounded by the XLA kernel launch overhead rather than compute latency. However, as the population scales into the massive regime ($10^4 \leq N \leq 10^7$), the GPU architecture becomes fully saturated. In this optimal compute-bound regime, Planax consistently sustains an unprecedented throughput exceeding 1.5×10^5 SPS. This represents an improvement of nearly two orders of magnitude over multi-core CPU baselines, fundamentally enabling the generation of billions of transition samples—required for stabilizing complex MARL—within hours instead of weeks.

2) *VRAM Memory Footprint*: Massive parallelism is only practical if the memory footprint remains tractable. Fig. 5 (Right) demonstrates the GPU VRAM consumption of Planax as a function of the agent population. Because the XLA compiler heavily fuses operations and eliminates the need to allocate intermediate tensors (e.g., instantaneous aerodynamic forces and RK4 stages) in global memory, the per-agent memory cost is drastically minimized.

The memory growth curve remains remarkably flat for $N < 10^5$. This is because the baseline VRAM is dominated by the static footprint of the compiled XLA computational graph and the tensorized F-16 aerodynamic parameters. Linear scaling is observed only at extreme population scales, allowing Planax to comfortably support up to 10^8 concurrent 6-DOF agents on a single 80GB A100 GPU before encountering Out-Of-Memory (OOM) limitations.

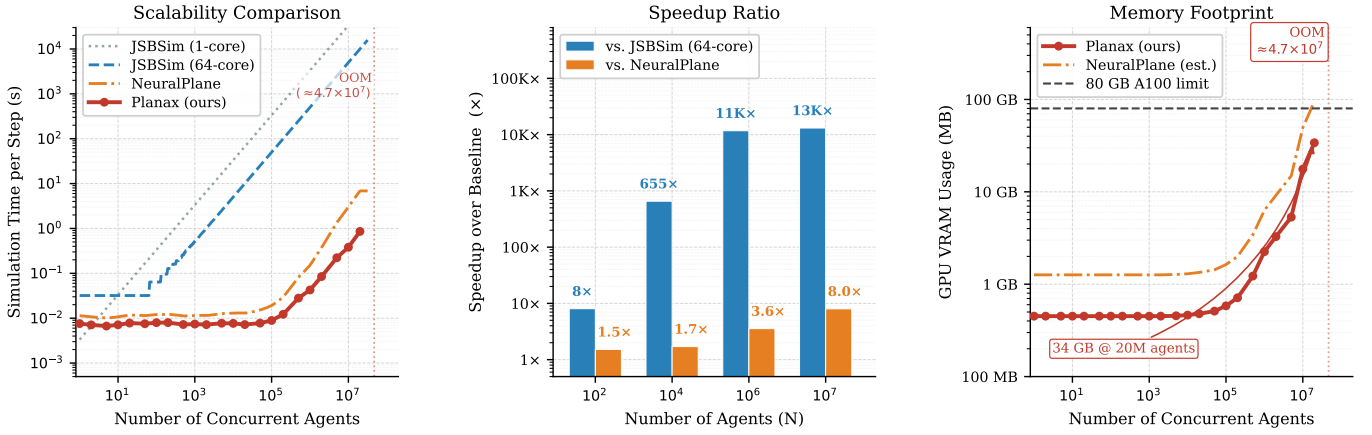


Fig. 5. **Scalability and Throughput Analysis.** (Left) Simulation time required versus the number of concurrent agents (log-log scale). Planax entirely avoids the exponential time growth seen in the CPU-centric JSBSim. (Middle) Speed-up ratio benchmark. At massive scales (e.g., 10^7 agents), Planax achieves an impressive $8\times$ acceleration over the PyTorch-based NeuralPlane, validating the efficiency of our XLA-fused, branch-free architecture. (Right) GPU memory footprint (VRAM). Planax exhibits a uniquely flat initial scaling curve dominated by the static compiled graph footprint. Its extreme memory efficiency seamlessly supports 10^7 concurrent high-fidelity agents on a single A100 GPU.

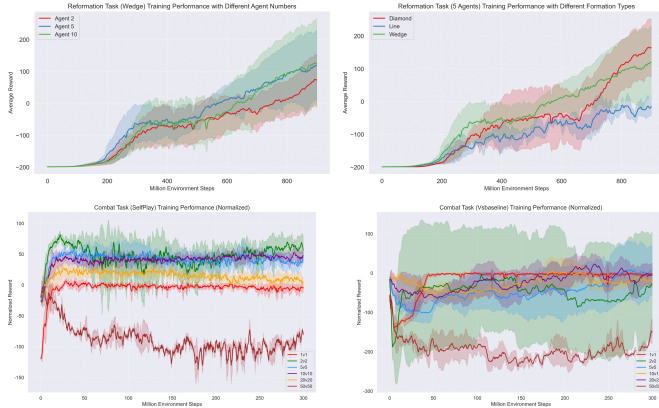


Fig. 6. **Learning Curves across Benchmark Tasks.** (Top) IPPO training returns for the decentralized formation keeping task across varying swarm sizes ($N = 2, 5, 10$) and geometric templates (Wedge, Diamond, Line). (Bottom) Hierarchical MAPPO returns in the adversarial pursuit-evasion benchmark under self-play and fixed-baseline settings, demonstrating successful convergence from 1-vs-1 up to massive 50-vs-50 engagements. Shaded regions denote the standard error across multiple random seeds.

B. Multi-Agent Benchmark Validation

Preliminary evaluations confirmed that individual agents readily converge on agile single-control tasks (e.g., heading tracking) under the F-16 6-DOF dynamics, laying the foundation for complex multi-agent interactions. Before analyzing the critical impact of physical fidelity (Section IV-C), we first validate that Planax provides a mathematically stable and efficient optimization landscape for standard MARL algorithms. We benchmarked the platform using Independent PPO (IPPO) [15] for decentralized cooperative tasks and a Hierarchical MAPPO (HRL) [16] architecture for adversarial tasks.

1) *Cooperative Formation Keeping:* Fig. 6 (Top) illustrates the training returns of the IPPO algorithm across varying swarm sizes and geometric templates (e.g., wedge, line, diamond). Despite the highly coupled 6-DOF aerodynamics and

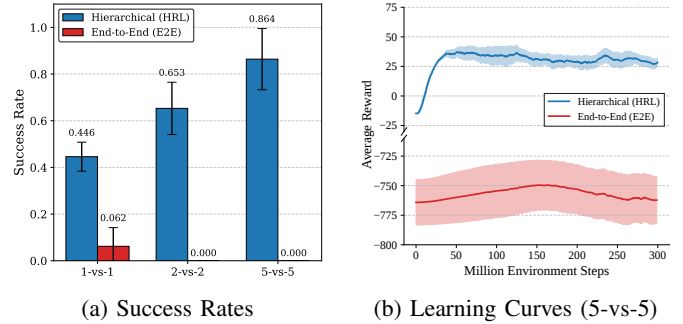


Fig. 7. **Impact of Hierarchical Policy Structure.** (Left) Comparison of success rates against a fixed-policy baseline. While HRL scales effectively with team size (reaching 86.4% in 5-vs-5), the E2E policy suffers a complete collapse in dense scenarios. (Right) Learning curves in the 5-vs-5 pursuit-evasion task, illustrating that the hierarchical decomposition is essential for stabilizing convergence in long-horizon, high-dynamic continuous control.

strict collision avoidance constraints, the decentralized policies demonstrate stable and monotonic convergence.

Notably, Planax successfully trains massive swarms (up to $N = 50$) within 3×10^8 environmental steps. Furthermore, zero-shot cross-scale transfer experiments (detailed in the Supplementary Material) reveal that policies trained on small sub-swarms ($N = 10$) generalize effectively to massive swarms ($N = 100$), maintaining over a 90% success rate. This confirms that the decentralized observation interface in Planax successfully encapsulates scalable, scale-invariant spatial coordination rules.

2) *Adversarial Pursuit-Evasion and Hierarchy:* In the highly non-stationary pursuit-evasion (combat) benchmark, we evaluated both self-play and fixed-baseline scenarios. As shown in Fig. 6 (Bottom), adversarial training introduces expected variance, yet the Centralized Training with Decentralized Execution (CTDE) paradigm successfully stabilizes the learning process.

A critical finding from our benchmark is the absolute necessity of hierarchical control in high-dynamic regimes. We

conducted an ablation study comparing our two-tier Hierarchical RL (high-level target generation, low-level continuous tracking) against a naive End-to-End (E2E) continuous control policy. As depicted in Fig. 7, while the E2E policy entirely fails to converge in dense multi-agent settings (e.g., 5-vs-5) due to the compounding complexity of long-horizon credit assignment and non-linear aerodynamic bounds, the HRL architecture rapidly converges to a high success rate. This validates that Planax’s abstraction perfectly supports modern structured decision-making algorithms required for real-world aviation.

C. Sim-to-Real Proxy: The Necessity of High-Fidelity

In the real world, trial-and-error for high-dynamic fixed-wing aerial robots is prohibitively expensive and physically catastrophic. To answer whether the massive computational effort required for 6-DOF high-fidelity aerodynamic modeling is truly necessary for MARL, we design a “Sim-to-Real Proxy” ablation study. We hypothesize that policies trained in simplified, low-fidelity (LoFi) kinematic environments exploit non-physical loopholes, rendering them undeployable. Therefore, our rigorous High-Fidelity (HiFi) engine serves as the ultimate “ground truth proxy” to safely evaluate real-world deployment viability.

1) *Ablation Design*: We utilize the Single-Agent Agile Control benchmark, requiring the F-16 to perform a continuous “S-maneuver” by tracking a target heading that alternates extreme angles every 10 seconds. Two policies are trained for 2,000 epochs: 1) **LoFi Policy**: Trained in a simplified dynamics environment, then deployed zero-shot into the HiFi environment. 2) **HiFi Policy**: Trained and evaluated in the same HiFi environment (domain matched). Both policies are evaluated across 200 parallel HiFi environments for a maximum horizon of 400 seconds.

2) *Quantitative Failure Analysis*: The evaluation yields a stark contrast, as summarized in Fig. 8. The **HiFi Policy** successfully adapts to the nonlinear coupling, maintaining an 85% survival rate through the 400-second episode with an average survival time of 340.2 seconds. It tracks the aggressive S-maneuver with a heading Root Mean Square Error (RMSE) of 49.35°.

Conversely, the **LoFi Policy** suffers a catastrophic 100% crash rate, surviving an average of merely 2.1 seconds before departing controlled flight. A deep analysis of the termination conditions reveals the root cause of this sim-to-real gap: the LoFi policy exhibits a G-overload violation rate of 1.40% per step (compared to 0.23% for the HiFi policy). Because the low-fidelity training environment did not penalize aggressive actuator inputs with realistic aerodynamic drag and inertia coupling, the LoFi policy learned dangerous, “shortcut” control behaviors. When subjected to the strict energy-maneuverability constraints of the HiFi environment, these commands immediately induce structural G-limit violations or accelerated stalls.

Because real-world fixed-wing crashes are catastrophic and costly, our high-fidelity simulator functions as an indispensable “safe trial-and-error proxy” for reality. This ablation unequivocally proves that incorporating high-fidelity aerodynamic constraints (e.g., branch-free tensorized LUTs) during training is

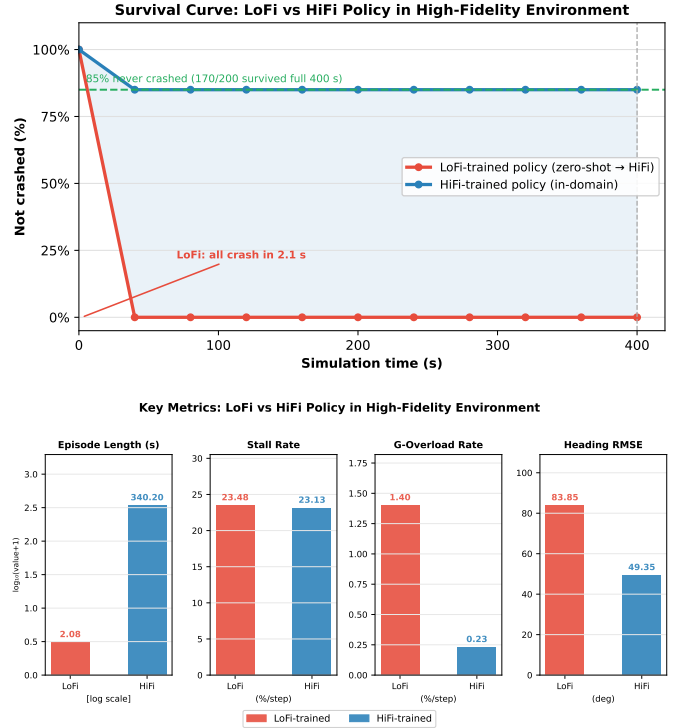


Fig. 8. **Sim-to-Real Proxy Validation.** (Top) Survival probability curves in the high-fidelity environment. The LoFi-trained policy suffers a 100% crash rate within average 2.1 seconds, while the Planax HiFi-trained policy maintains an 85% survival rate. (Bottom) Quantitative comparison of key physical metrics. The LoFi policy’s failure is driven by significantly higher stall rates (2.08%/step) and G-overload violations (1.40%/step) when subjected to realistic aerodynamic constraints.

not merely an optional enhancement, but an absolute necessity for generating deployable and safe flight policies for high-dynamic aerial robots.

V. CONCLUSION

This letter presented Planax, a massively parallel, JAX-accelerated simulation framework designed to bridge the gap between high-fidelity aerodynamics and scalable MARL for fixed-wing aerial swarms. By formulating a 6-DOF nonlinear flight dynamics engine with tensorized aerodynamic lookup tables into a fully GPU-resident zero-transfer loop, Planax completely bypasses traditional PCIe memory bottlenecks. Our evaluations demonstrate that the platform not only supports unprecedented simulation throughput—scaling up to 10^8 concurrent agents—but also provides a stable landscape for complex multi-agent algorithms, such as hierarchical policy optimization in adversarial environments.

Most importantly, through a rigorous sim-to-real proxy ablation, we proved that high-fidelity aerodynamic constraints are absolutely essential for agile flight. Lacking them, policies inevitably exploit non-physical shortcuts, leading to catastrophic structural failures (e.g., fatal G-overloads) during high-dynamic maneuvers. Future work will expand Planax to encompass multi-fidelity heterogeneous airframes and complex atmospheric disturbances (e.g., wake turbulence), further paving the way for the direct deployment of massive aerial intelligence to real-world physical swarms.

REFERENCES

- [1] J. S. Berndt, “Jsbsim: An open source flight dynamics model in C++,” in *AIAA Modeling and Simulation Technologies Conference*, 2004, p. 4923.
- [2] Laminar Research, “X-plane flight simulator,” <https://www.x-plane.com/>, 2025.
- [3] J. Panerati, H. Zheng, S. Zhou, J. Dong, Z. Grey, and A. Schoellig, “Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control,” *IEEE Robotics and Automation Letters*, vol. 7, no. 1, pp. 1228–1235, 2021.
- [4] V. Makoviychuk and et al., “Isaac Gym: High performance GPU-based physics simulation for robot learning,” 2021.
- [5] C. Xue and et al., “Neuralplane: An efficiently parallelizable platform for fixed-wing aircraft,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [6] J. Bradbury and et al., “JAX: Composable transformations of Python+NumPy programs,” <https://github.com/google/jax>, 2018.
- [7] D. Freeman and et al., “Brax: A differentiable physics engine for large scale rigid body simulation,” 2021.
- [8] A. Rutherford and et al., “JaxMARL: A suite of multi-agent reinforcement learning environments and algorithms in JAX,” 2023.
- [9] R. Lowe and et al., “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [10] M. Samvelyan and et al., “The StarCraft multi-agent challenge,” in *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2019, pp. 2186–2188.
- [11] C. Richter and et al., “QPlane: An open-source reinforcement learning toolkit for autonomous fixed wing aircraft simulation,” in *Proceedings of the ACM Symposium on Applied Computing (SAC)*, 2021, pp. 1065–1072.
- [12] A. Pope and et al., “Hierarchical reinforcement learning for air combat,” *IEEE Transactions on Artificial Intelligence*, vol. 3, no. 6, pp. 878–889, 2022.
- [13] Y. Jiang and et al., “Learning agile fighter maneuvers using hierarchical multi-agent reinforcement learning,” *IEEE Transactions on Neural Networks and Learning Systems*, 2024, early Access.
- [14] B. L. Stevens, F. L. Lewis, and E. N. Johnson, *Aircraft Control and Simulation*, 3rd ed. John Wiley & Sons, 2015.
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017.
- [16] C. Yu and et al., “The surprising effectiveness of PPO in cooperative multi-agent games,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 611–24 624.